

Docker 1..10

Instrucciones

Resuelve los ejercicios de Docker que se presentan a continuación.

Entrega un único PDF con capturas, organizado por número de ejercicio. Incluye los cambios realizados, los comandos utilizados y el resultado de las comprobaciones que se indican en cada apartado.

Preparación

Para los ejercicios 1 a 5, utiliza la aplicación de lista de tareas [getting-started-todo-app](#) trabajada en clase. Arráncala con Docker Compose y activa la sincronización de cambios con `docker compose watch` o `docker compose up --watch`. Comprueba que puedes abrir la aplicación en el navegador antes de modificarla.

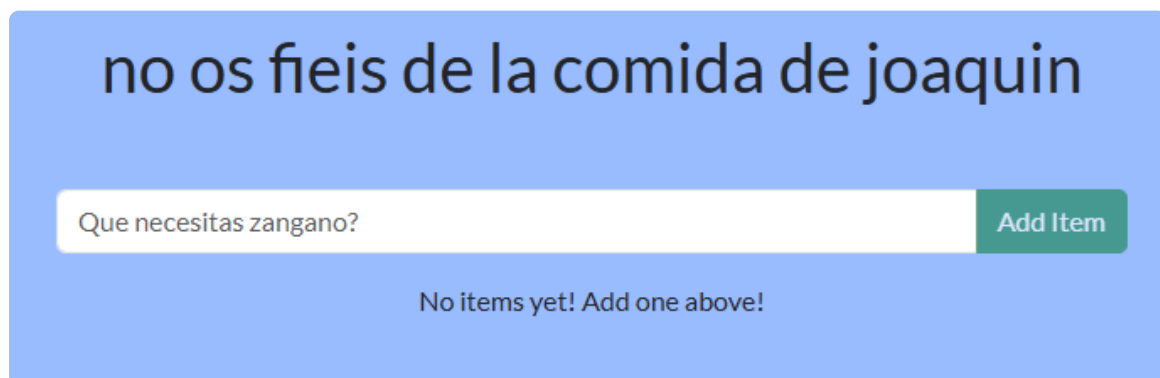
EJERCICIO 01

Mostrar saludos aleatorios

[Ver diapositiva 36](#) ➤

En `backend/src/routes/getGreeting.js`, sustituye el saludo fijo por un array llamado `greetings` con varios mensajes. Selecciona un saludo de forma aleatoria en cada petición utilizando `Math.floor()` y `Math.random()`.

Comprobación: recarga la aplicación varias veces y documenta que puede mostrar distintos saludos. Incluye una captura del código modificado y capturas del resultado en el navegador.



Ejemplo de la aplicación con el saludo personalizado. [Ver diapositiva 36](#) ➤

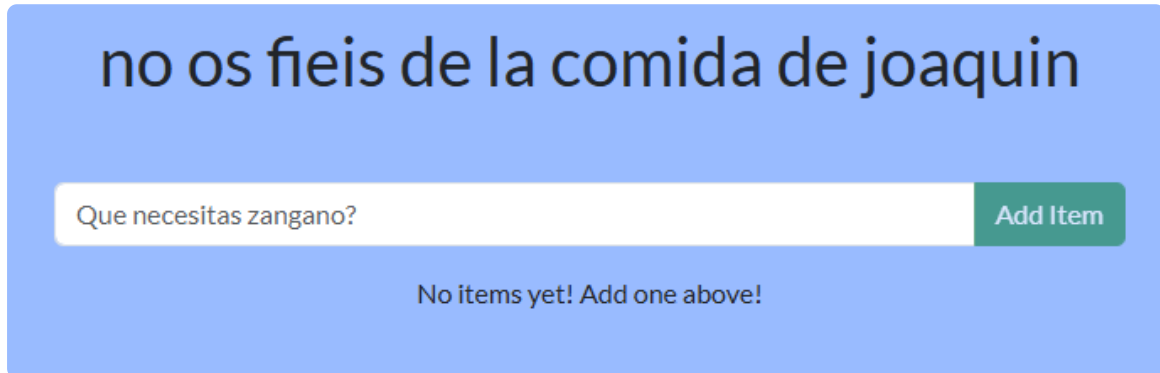
EJERCICIO 02

Cambiar el texto del formulario

[Ver diapositiva 37 ↗](#)

Abre `client/src/components/AddNewItemForm.jsx` y cambia el atributo `placeholder` del campo donde se escribe una nueva tarea por el texto «**Que necesitas zangano?**».

Comprobación: muestra el formulario con el campo vacío para que se vea el nuevo texto. Incluye una captura del cambio en el archivo y otra de la aplicación.

A screenshot of a web application interface. At the top, there is a large heading "no os fieis de la comida de joaquin" in a dark font. Below this, there is a light blue rectangular container. Inside this container, there is a white input field with the placeholder text "Que necesitas zangano?". To the right of the input field is a green button with the text "Add Item". Below the input field and button, there is a line of text that says "No items yet! Add one above!".

Referencia del nuevo texto del formulario. [Ver diapositiva 37 ↗](#)

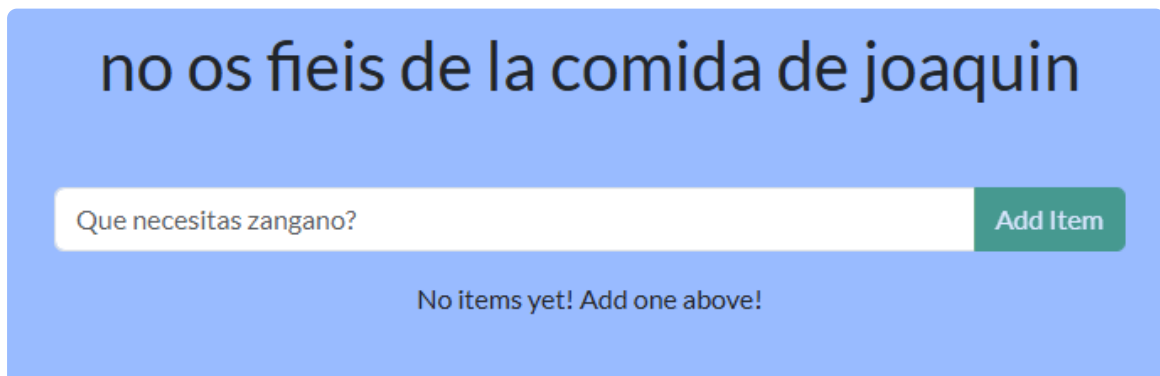
EJERCICIO 03

Personalizar el color de fondo

[Ver diapositiva 38 ↗](#)

En `client/src/index.scss`, modifica la propiedad `background-color` para utilizar un color de tu elección.

Comprobación: muestra el valor elegido en el archivo y el nuevo fondo de la aplicación en el navegador.

A screenshot of the same web application interface as in the previous image. The background of the entire page is now a solid light blue color. The heading "no os fieis de la comida de joaquin" and the form container are still present, with the input field showing the placeholder "Que necesitas zangano?" and the "Add Item" button.

Ejemplo de personalización del fondo. [Ver diapositiva 38 ↗](#)

EJERCICIO 04

Publicar una imagen desde la terminal

[Ver diapositiva 46](#) ➤

Construye una imagen de la aplicación modificada y publícala en tu cuenta de Docker Hub utilizando comandos. El repositorio de la imagen debe llamarse ejercicio4, con el formato tu_usuario/ejercicio4.

Comprobación: adjunta capturas del proceso de construcción, de la imagen disponible en tu equipo, de su subida y del resultado en Docker Hub. Identifica los comandos que has utilizado.

```
PS C:\Users\diego> docker image ls
REPOSITORY                                TAG      IMAGE ID      CREATED        SIZE
getting-started-todo-app-main-backend     latest   e5bc2fdc384a  About an hour ago  1.28GB
getting-started-todo-app-main-client      latest   537e502979ef  About an hour ago  1.25GB
diegoayaladocker/docker-quickstart       latest   1828a97621f9  6 days ago     141MB
diegoayaladocker/getting-started-todo-app latest   53cbb5966952  13 days ago    1.21GB
<none>                                    <none>   fd775413e9fa  2 weeks ago    1.25GB
<none>                                    <none>   9166401c7c2f  2 weeks ago    1.24GB
docker/welcome-to-docker                 latest   6caf772f5178  2 months ago   14.1MB
traefik                                   v3.4     aa215d9f973d  2 months ago   227MB
mysql                                      9.3      850100bac3be  5 months ago   859MB
phpmyadmin                                latest   2d9a07a01a58  8 months ago   568MB
```

Referencia de la lista de imágenes disponibles en el equipo. [Ver diapositiva 44](#) ➤

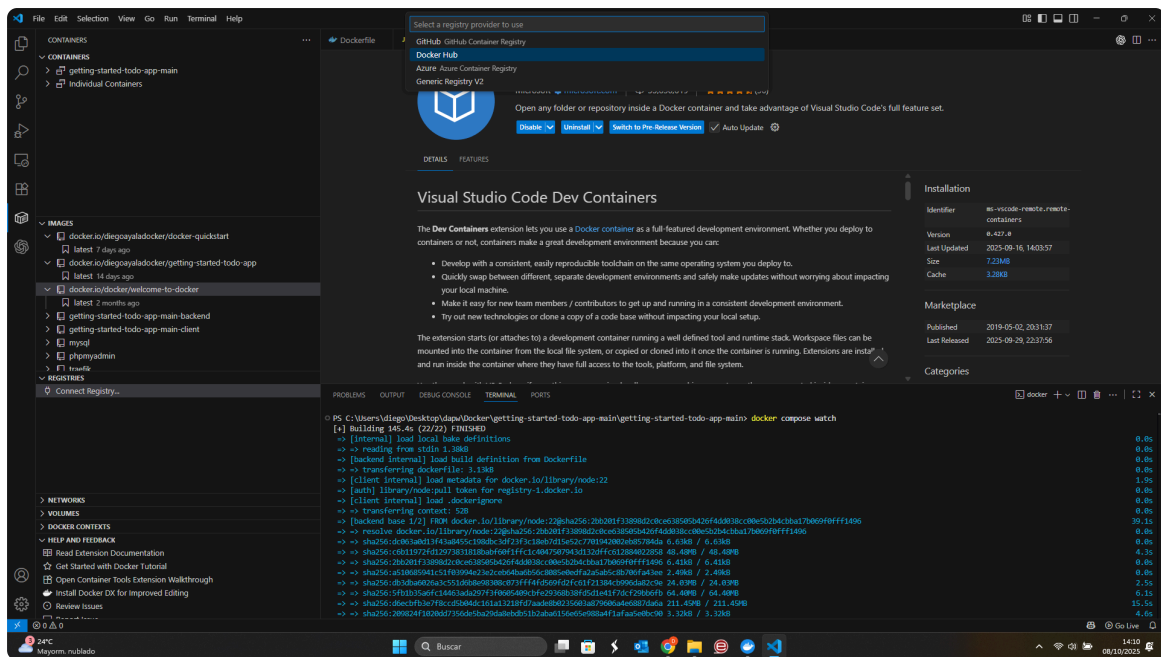
EJERCICIO 05

Publicar una imagen desde VS Code

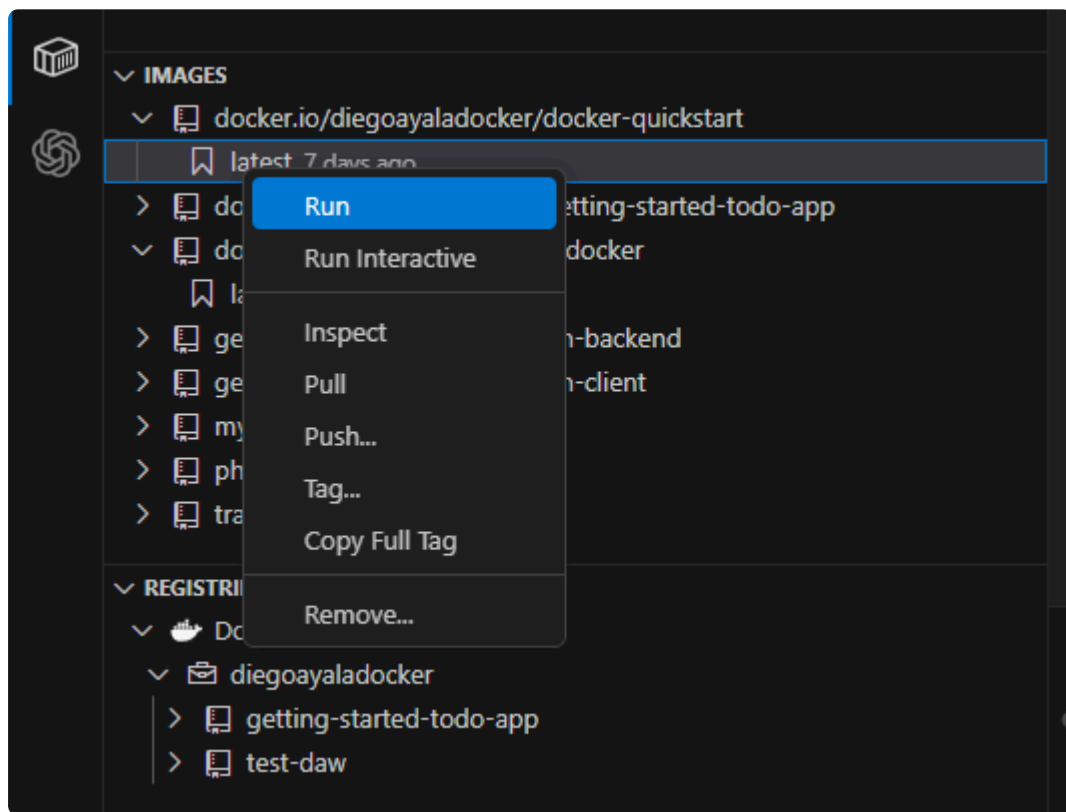
[Ver diapositiva 47](#) ➤

Publica la imagen de la aplicación en Docker Hub utilizando las herramientas de contenedores de Visual Studio Code. Realiza la subida desde la interfaz de VS Code.

Comprobación: adjunta el resultado final de la publicación e identifica el nombre y la etiqueta de la imagen. Incluye una captura de la operación en VS Code y otra de la imagen publicada en Docker Hub.



Conexión a Docker Hub desde VS Code. [Ver diapositiva 42 ↗](#)



La opción Push permite publicar la imagen desde VS Code. [Ver diapositiva 42 ↗](#)

EJERCICIO 06

Mantener activo un contenedor Alpine

[Ver diapositiva 72](#) ➤

Partiendo de la imagen `alpine`, pasa de un contenedor que ejecuta una tarea puntual y termina (*one-off*) a uno que permanezca activo y en el que puedas ejecutar comandos.

Comprobación: muestra cómo lo arrancas, ejecuta varios comandos dentro del contenedor y comprueba su estado desde otra terminal. Explica brevemente qué mantiene el contenedor en ejecución.

EJERCICIO 09

Explorar imágenes de Java

[Ver diapositiva 74](#) ➤

Busca en Docker Hub una imagen que permita trabajar con Java. Ejecuta un contenedor y comprueba la versión instalada con `java -version`.

Después, localiza una imagen más pequeña que incluya únicamente el entorno de ejecución JRE. Identifica ambas imágenes y sus etiquetas, compara sus tamaños y explica la diferencia entre disponer de un JDK y de un JRE.

Comprobación: incluye las referencias de Docker Hub, una captura de la versión de Java dentro del contenedor y otra que permita comparar el tamaño de las imágenes descargadas.

EJERCICIO 10

Sumar dos números en Java con Docker

[Ver diapositiva 75](#) ➤

Crea un programa `Suma.java` que pida dos números por teclado, calcule su suma y muestre el resultado por pantalla.

1. **Prueba en local.** Compila el archivo con `javac Suma.java` y ejecútalo con `java Suma`. Comprueba que solicita los dos números y muestra la suma correcta.
2. **Prepara el Dockerfile.** En el mismo directorio, crea un `Dockerfile` que parta de una imagen de Java encontrada en Docker Hub, copie `Suma.java`, lo compile dentro de la imagen y establezca la ejecución del programa al iniciar el contenedor. La imagen elegida debe incluir el compilador `javac`.
3. **Construye la imagen.** Asigna un nombre a tu imagen y comprueba que la construcción termina correctamente.
4. **Ejecuta el contenedor.** Arráncalo de forma que puedas introducir los dos números por teclado. Verifica que muestra el mismo resultado que la ejecución local.

Comprobación: incluye el código de `Suma.java`, el contenido del `Dockerfile` y capturas de la compilación y ejecución local, de la construcción de la imagen y de la suma realizada dentro del contenedor.

[Consultar el documento original del ejercicio 10.](#)

Materiales de la tarea

Los archivos adjuntos se descargan por separado desde la página de esta tarea.



Diapositivas de Docker · DAPW

Presentación de clase · abrir desde el ejercicio 1

